O'REILLY LIVE TRAINING · 2 HOURS

Claude Certified Architect – Foundations Exam Prep

Cut to the chase for CCAR-F: the blueprint, the six scenarios, and the traps that cost points.

Today's plan

SEGMENT	CLOCK	DOMAINS	WEIGHT
1 · The exam + agentic architecture	0:00–0:30	Blueprint + D1	27%
2 · Tools, MCP, Claude Code	0:30–1:05	D2 + D3	38%
3 · Prompting, output, context	1:05–1:50	D4 + D5	35%
4 · Exam strategy + study plan	1:50–2:00	All	·

Twelve official sample questions · four official prep exercises.

SEGMENT 1 · 0:00 → 0:30 · 30 MIN

The exam and agentic architecture

Exam blueprint · 5 domains · 6 scenarios

D1 Agentic Architecture & Orchestration · **27%**

Exam guide §3–§6

Prep exercise 1

The exam at a glance

ITEM	DETAIL
Format	**60 items** · multiple-choice + multiple-response ("select N")
Structure	**4 scenarios** drawn at random from a bank of **6**
Time	120 min · **2 min per item**
Pass	Scaled **720** on 100–1,000 · 720 ≠ 72% raw
Delivery	**$125** · Pearson VUE, online or test center
Validity	12 months · free on-time renewal

WHO THE EXAM IS WRITTEN FOR

A solution architect who builds with Claude

> *"a solution architect who designs and implements production applications with Claude."*

- Builds agents with the **Agent SDK**: loops, subagents, tools, hooks
- Configures **Claude Code** for teams: CLAUDE.md, skills, MCP, plan mode
- Designs **MCP** tools, **structured output**, and **context** strategies
- Wires Claude into **CI/CD** and makes escalation and reliability calls
- Recommended: **6+ months** building with Claude APIs, Agent SDK, Claude Code, and MCP

The blueprint: five weighted domains

#	DOMAIN	WEIGHT	≈ ITEMS	TASK STATEMENTS
D1	**Agentic Architecture & Orchestration**	**27%**	**16**	7
D2	Tool Design & MCP Integration	18%	11	5
D3	Claude Code Configuration & Workflows	20%	12	6
D4	Prompt Engineering & Structured Output	20%	12	6
D5	Context Management & Reliability	15%	9	6

D1 + D3 + D4 = 67%. Orchestrate agents, configure Claude Code, get reliable output.

Six scenarios, four on your exam

SCENARIO	PRIMARY DOMAINS
1 · Customer Support Resolution Agent	D1 · D2 · D5
2 · Code Generation with Claude Code	D3 · D5
3 · Multi-Agent Research System	D1 · D2 · D5
4 · Developer Productivity with Claude	D2 · D3 · D1
5 · Claude Code for Continuous Integration	D3 · D4
6 · Structured Data Extraction	D4 · D5

Study all six. You won't know which four you'll get.

What's in scope, what isn't

TESTED

- Agentic loops, `stop_reason`, hooks
- Coordinator–subagent patterns
- Tool descriptions, MCP config, errors
- CLAUDE.md, rules, commands, skills
- `tool_use` + JSON schema, `tool_choice`
- Batches API, few-shot, context, provenance

NOT TESTED

- Fine-tuning, training, RLHF, Constitutional AI
- Auth, billing, rate limits, pricing math
- Hosting MCP servers, cloud provider config
- Embeddings, vector DBs, vision, computer use
- Streaming / SSE, tokenization
- Prompt caching "beyond knowing it exists"

Six distractor patterns

PATTERN	SOUNDS LIKE
Prompt as policy	"Make it mandatory in the system prompt."
Over-engineering	"Add a routing classifier." "Train a model on old tickets."
Bigger model	"Higher-tier model with a larger context window."
Unreliable proxy	"Self-reported confidence 1–10." "Sentiment analysis."
Invented feature	`CLAUDE_HEADLESS=true` · `--batch` · `.claude/config.json`
Hide or kill the error	"Return empty as success." "Terminate the workflow."

Right answer: the proportionate fix at the root cause.

What the exam asks

- 1.1 Design and implement agentic loops for autonomous task execution
- 1.2 Orchestrate multi-agent systems with coordinator-subagent patterns
- 1.3 Configure subagent invocation, context passing, and spawning
- 1.4 Implement multi-step workflows with enforcement and handoff patterns
- 1.5 Apply Agent SDK hooks for tool call interception and data normalization
- 1.6 Design task decomposition strategies for complex workflows
- 1.7 Manage session state, resumption, and forking

Task statements verbatim from the CCAR-F Exam Guide v1.0, Section 6

The agentic loop runs on `stop_reason`

```
while True:
    resp = client.messages.create(model=MODEL, tools=tools, messages=messages)
    messages.append({"role": "assistant", "content": resp.content})
    if resp.stop_reason ≠ "tool_use":      # "end_turn" → done
        break
    results = [run_tool(b) for b in resp.content if b.type == "tool_use"]
    messages.append({"role": "user", "content": results})
```

ANTI-PATTERN	WHY IT FAILS
Parse the text for "done"	Natural language is not a control signal
Iteration cap as the main stop	Cuts off valid work; use it only as a safety net
"Has text content" = finished	Claude can emit text *and* call tools

Coordinator and subagents: hub-and-spoke

THE COORDINATOR

- Decomposes, delegates, aggregates
- Picks subagents by query complexity
- Routes **all** inter-agent traffic
- Sets goals + quality criteria, not steps
- Re-delegates when synthesis shows gaps

THE SUBAGENTS

- Spawned via the `Task` tool
- `allowedTools` must include `"Task"`
- Isolated context: **no inherited history**
- Get prior findings pasted into the prompt
- Parallel = many `Task` calls in **one** response

Subagents start with only what you pass them. Use structured data so source URLs and page numbers survive.

Enforce in code, not in the prompt

NEED	MECHANISM
Verify identity before refunds	**Prerequisite gate**: block `process_refund` until `get_customer` returns a verified ID
Block refunds over $500	**Tool-call interception hook** → redirect to human escalation
Normalize Unix / ISO 8601 / status codes	**PostToolUse hook** transforms results before the model sees them
Hand off to a human mid-process	**Structured summary**: customer ID, root cause, amount, recommended action
Multi-concern request	Split into items, investigate in parallel, synthesize one answer

Prompt instructions have a non-zero failure rate. Hooks are deterministic.

Decompose and manage sessions

SITUATION	CHOOSE
Predictable multi-aspect review	**Prompt chaining**: fixed sequential passes
Open-ended investigation	**Dynamic decomposition**: map, prioritize, adapt
Big code review	Per-file local passes + one cross-file integration pass
Continue yesterday's investigation	`--resume <session-name>`
Compare two approaches from one baseline	`fork_session`
Prior tool results are stale	New session + injected structured summary
Files changed since last session	Resume and name the changed files

Production data shows that in 12% of cases, your agent skips get_customer entirely and calls lookup_order using only the customer's stated name, occasionally leading to misidentified accounts and incorrect refunds. What change would most effectively address this reliability issue?

A. Add a programmatic prerequisite that blocks lookup_order and process_refund calls until get_customer has returned a verified customer ID.

B. Enhance the system prompt to state that customer verification via get_customer is mandatory before any order operations.

C. Add few-shot examples showing the agent always calling get_customer first, even when customers volunteer order details.

D. Implement a routing classifier that analyzes each request and enables only the subset of tools appropriate for that request type.

A · Programmatic prerequisite gate

OPTION	PATTERN	WHY IT FAILS
B	Prompt as policy	Probabilistic compliance; errors cost money
C	Prompt as policy	Few-shot is still probabilistic
D	Over-engineering	Fixes tool availability, not tool ordering

Required sequence + financial stakes → enforce in code.

On the topic "impact of AI on creative industries," every subagent completes successfully, but the final reports cover only visual arts, missing music, writing, and film. The coordinator's logs show it decomposed the topic into "AI in digital art creation," "AI in graphic design," and "AI in photography." What is the most likely root cause?

A. The synthesis agent lacks instructions for identifying coverage gaps in the findings it receives.

B. The coordinator agent's task decomposition is too narrow, so subagent assignments don't cover all relevant domains.

C. The web search agent's queries are not comprehensive enough and need to cover more sectors.

D. The document analysis agent filters out non-visual sources due to overly restrictive relevance criteria.

B · The coordinator's decomposition is too narrow

OPTION	PATTERN	WHY IT FAILS
A	Blame downstream	Synthesis worked on what it was given
C	Blame downstream	Search covered its assigned subtopics
D	Blame downstream	Analysis worked within its assigned scope

Read the logs the stem gives you. The fault sits where the scope was set.

OFFICIAL PREP EXERCISE 1 · ~6 MIN

Multi-tool agent with escalation logic

- **Build:** 3–4 MCP tools, two of them deliberately similar; a loop on `stop_reason`
- **Add:** structured errors (`errorCategory`, `isRetryable`) and a hook that blocks over-threshold refunds → escalation
- **Test:** a multi-concern message; check it decomposes, handles each issue, and answers once
- **Reinforces:** D1 · D2 · D5

SEGMENT 1 COMPLETE

Q&A

SEGMENT 2 · 0:30 → 1:05 · 35 MIN

Tools, MCP, and Claude Code

D2 Tool Design & MCP Integration · **18%**

D3 Claude Code Configuration & Workflows · **20%**

Exam guide §6

Prep exercise 2

What the exam asks

- 2.1 Design effective tool interfaces with clear descriptions and boundaries
- 2.2 Implement structured error responses for MCP tools
- 2.3 Distribute tools appropriately across agents and configure tool choice
- 2.4 Integrate MCP servers into Claude Code and agent workflows
- 2.5 Select and apply built-in tools (Read, Write, Edit, Bash, Grep, Glob) effectively

Task statements verbatim from the CCAR-F Exam Guide v1.0, Section 6

The description is the router

PROBLEM	FIX
Minimal descriptions, similar tools	Add input formats, example queries, edge cases, "use X instead when..."
Near-identical names (`analyze_content` vs `analyze_document`)	**Rename** and rescope: `extract_web_results`
One generic tool doing three jobs	**Split**: `extract_data_points` · `summarize_content` · `verify_claim_against_source`
Agent prefers Grep over your MCP tool	Describe the MCP tool's capabilities and outputs in detail
System prompt keywords pull the wrong tool	Review the prompt for keyword-sensitive instructions

Production logs show the agent frequently calls get_customer when users ask about orders (e.g., "check my order #12345"), instead of calling lookup_order. Both tools have minimal descriptions ("Retrieves customer information" / "Retrieves order details") and accept similar identifier formats. What's the most effective first step to improve tool selection reliability?

A. Add 5–8 few-shot examples to the system prompt showing order-related queries routing to lookup_order.

B. Expand each tool's description to include input formats, example queries, edge cases, and boundaries versus similar tools.

C. Implement a routing layer that parses user input and pre-selects the tool based on keywords and identifier patterns.

D. Consolidate both tools into a single lookup_entity tool that internally determines which backend to query.

B · Expand the tool descriptions

OPTION	PATTERN	WHY IT FAILS
A	Symptom patch	Token overhead; root cause stays
C	Over-engineering	Bypasses the model's language understanding
D	Too big a first step	Valid design, but more effort than the problem needs

"First step" + root cause in the stem → the cheapest fix that removes it.

Errors the agent can act on

CATEGORY	EXAMPLE	AGENT SHOULD	RETRYABLE?
Transient	Timeout, service down	Retry (locally first)	`true`
Validation	Bad input format	Fix the input, call again	`false`
Business	Refund exceeds policy	Explain to the customer	`false`
Permission	No access	Escalate	`false`

```
{"isError": true, "errorCategory": "business", "isRetryable": false,
 "message": "Refunds over $500 need manager approval."}
```

An empty result is a success. Don't confuse "no matches" with "couldn't reach the source".

Give each agent fewer, sharper tools

RULE	EXAMPLE
4–5 tools, not 18	Too many tools degrade selection
Scope tools to the role	A synthesis agent shouldn't run web searches
Scoped cross-role tool for the common case	`verify_fact` for synthesis; hard cases still go via the coordinator
Constrain generic tools	`fetch_url` → `load_document` that validates URLs
`tool_choice: "auto"`	Model may answer in text
`tool_choice: "any"`	Must call *a* tool
`{"type": "tool", "name": "…"}`	Must call *this* tool (e.g., `extract_metadata` first)

MCP in Claude Code

PROJECT · `.mcp.json`

- Shared team tooling, in version control
- Secrets via `${GITHUB_TOKEN}` expansion
- Prefer a community server for Jira-style integrations

USER · `~/.claude.json`

- Personal or experimental servers
- Not shared with teammates
- Custom servers only for team-specific workflows

- Tools from **all** configured servers are discovered at connect time and available at once
- **Resources** expose catalogs (issues, docs, schemas) so the agent skips exploratory calls

What the exam asks

- 3.1 Configure CLAUDE.md files with appropriate hierarchy, scoping, and modular organization
- 3.2 Create and configure custom slash commands and skills
- 3.3 Apply path-specific rules for conditional convention loading
- 3.4 Determine when to use plan mode vs direct execution
- 3.5 Apply iterative refinement techniques for progressive improvement
- 3.6 Integrate Claude Code into CI/CD pipelines

Task statements verbatim from the CCAR-F Exam Guide v1.0, Section 6

Where does it go?

NEED	PUT IT IN
My personal preferences	`~/.claude/CLAUDE.md` (**not** shared)
Team standards, always loaded	Root `CLAUDE.md` or `.claude/CLAUDE.md`
One package's conventions	Subdirectory `CLAUDE.md` + `@import` of its standards files
Conventions by file type, any folder	`.claude/rules/*.md` with `paths:` globs
Team slash command	`.claude/commands/`
On-demand workflow	`.claude/skills/<name>/SKILL.md`
Verbose skill output kept out of main chat	`context: fork` in SKILL.md frontmatter
Which memory files loaded?	`/memory`

Your codebase has distinct areas with different conventions: React components, API handlers, and database models. Test files sit next to the code they test (Button.test.tsx next to Button.tsx), and you want all tests to follow the same conventions regardless of location. What's the most maintainable way to ensure Claude automatically applies the correct conventions when generating code?

A. Create rule files in .claude/rules/ with YAML frontmatter glob patterns that apply conventions based on file paths.

B. Consolidate all conventions in the root CLAUDE.md under headers for each area, relying on Claude to infer which applies.

C. Create skills in .claude/skills/ for each code type with the conventions in their SKILL.md files.

D. Place a separate CLAUDE.md file in each subdirectory containing that area's conventions.

A · .claude/rules/ with glob patterns

OPTION	WHY IT FAILS
B	Relies on inference, not explicit matching
C	Skills load on demand, not automatically by path
D	CLAUDE.md is directory-bound; tests live everywhere

```
# .claude/rules/testing.md frontmatter
paths: ["**/*.test.tsx"]
```

Plan first, or just do it?

PLAN MODE

- Architectural decisions
- Multiple valid approaches
- Migrations touching 45+ files
- Monolith → microservices

DIRECT EXECUTION

- Single-file fix with a clear stack trace
- Add one validation check
- Scope already well understood
- Execute a plan you already made

Iterate with evidence: 2–3 input/output examples · tests first, then share failures · let Claude interview you · one message for interacting issues, one at a time for independent ones.

Claude Code in CI

```
claude -p "Review this PR for security issues" \
  --output-format json --json-schema review-schema.json
```

- `-p` / `--print`: non-interactive. Without it, the job **hangs** waiting for input
- `--output-format json` + `--json-schema` → findings you can post as inline PR comments
- **CLAUDE.md** carries testing standards, fixtures, and review criteria into CI
- Review with an **independent instance**, not the session that wrote the code
- On re-runs, pass prior findings and report only new or unaddressed issues
- Pass existing tests so generation skips covered scenarios

Q4. A /review slash command must be available to every developer who clones the repo. Where does the file go? **A** `.claude/commands/` · **B** `~/.claude/commands/` · **C** root CLAUDE.md · **D** `.claude/config.json`

Q5. Restructure a monolith into microservices: dozens of files, decisions about service boundaries. **A** plan mode first · **B** direct execution, let boundaries emerge · **C** direct execution with detailed instructions · **D** direct, switch to plan if it gets hard

Q10. `claude "Analyze this PR for security issues"` hangs in CI waiting for input. **A** add `-p` · **B** `CLAUDE_HEADLESS=true` · **C** redirect stdin from `/dev/null` · **D** add `--batch`

Q	ANSWER	TRAP IN THE DISTRACTORS
4	**A** `.claude/commands/`	B is personal; D doesn't exist
5	**A** Plan mode	D ignores complexity the stem already states
10	**A** `-p`	B and D are invented features

Know the real flags and paths. Invented ones are free points.

OFFICIAL PREP EXERCISE 2 · ~6 MIN

Configure Claude Code for a team

- **Create:** project CLAUDE.md; `.claude/rules/` with `paths: ["src/api/**/*"]` and `paths: ["**/*.test.*"]`
- **Add:** a project skill with `context: fork` + `allowed-tools`; `.mcp.json` with env-var secrets; a personal server in `~/.claude.json`
- **Compare:** plan mode vs direct on a one-file fix, a multi-file migration, and a feature with several valid designs
- **Reinforces:** D3 · D2

SEGMENT 2 COMPLETE

Q&A + 5-minute break

SEGMENT 3 · 1:05 → 1:50 · 45 MIN

Prompting, structured output, and context

D4 Prompt Engineering & Structured Output · **20%**

D5 Context Management & Reliability · **15%**

Exam guide §6

Prep exercises 3 + 4

What the exam asks

- 4.1 Design prompts with explicit criteria to improve precision and reduce false positives
- 4.2 Apply few-shot prompting to improve output consistency and quality
- 4.3 Enforce structured output using tool use and JSON schemas
- 4.4 Implement validation, retry, and feedback loops for extraction quality
- 4.5 Design efficient batch processing strategies
- 4.6 Design multi-instance and multi-pass review architectures

Task statements verbatim from the CCAR-F Exam Guide v1.0, Section 6

Explicit criteria, then examples

VAGUE (FAILS)	EXPLICIT (WORKS)
"Check that comments are accurate"	"Flag a comment only when its claimed behavior contradicts the code"
"Be conservative"	Report bugs and security; skip minor style and local patterns
"Only high-confidence findings"	Severity levels, each with a concrete code example

Few-shot: 2–4 targeted examples for the ambiguous cases, showing *why* one action beat the alternatives. They fix format drift, cut false positives, and reduce empty or invented extraction fields.

False positives in one category erode trust in all of them. Disable that category while you fix its prompt.

Schema-valid isn't the same as correct

```
{"name": "extract_invoice", "input_schema": {"type": "object",
  "properties": {
    "stated_total":     {"type": "number"},
    "calculated_total": {"type": "number"},
    "due_date":         {"type": ["string", "null"]},
    "category": {"enum": ["services", "goods", "other", "unclear"]},
    "category_detail":  {"type": ["string", "null"]}},
  "required": ["stated_total", "calculated_total", "category"]}}
```

- `tool_use` + JSON schema **removes syntax errors**, not semantic ones
- **Nullable** fields stop the model inventing values to fill required ones
- `"other"` + detail and `"unclear"` keep categories honest
- Unknown doc type → `tool_choice: "any"`; must run first → force the tool

Retry only what a retry can fix

FAILURE	RETRY HELPS?	DO
Format mismatch	**Yes**	Resend document + failed output + the specific validation error
Structural output error	**Yes**	Same, with the error message
Totals don't add up	Sometimes	Extract `calculated_total` beside `stated_total`; flag conflicts
Info only in a document you didn't send	**No**	Return null or route to a human
Developers keep dismissing a finding	n/a	Log `detected_pattern` and analyze the dismissals

Batch what can wait

	SYNCHRONOUS API	MESSAGE BATCHES API
Cost	Standard	**50% off**
Latency	Seconds	Up to **24 h**, no SLA
Multi-turn tool calls	Yes	**No**
Fits	Blocking pre-merge checks	Overnight reports, weekly audits, nightly tests
Correlate results	n/a	`custom_id`

- Resubmit **only failures** by `custom_id` (chunk the oversized ones)
- Refine the prompt on a sample **before** the big batch
- 30 h SLA with 24 h processing → submit every **4 h** or sooner

Your team wants to reduce API costs. Real-time Claude calls power two workflows: (1) a blocking pre-merge check that must complete before developers can merge, and (2) a technical debt report generated overnight for review the next morning. Your manager proposes switching both to the Message Batches API for its 50% cost savings. How should you evaluate this proposal?

A. Use batch processing for the technical debt reports only; keep real-time calls for pre-merge checks.

B. Switch both workflows to batch processing with status polling to check for completion.

C. Keep real-time calls for both workflows to avoid batch result ordering issues.

D. Switch both to batch processing with a timeout fallback to real-time if batches take too long.

A · Batch the overnight report only

OPTION	PATTERN	WHY IT FAILS
B	Wrong trigger	No latency SLA for a blocking check
C	Misconception	`custom_id` correlates results; order doesn't matter
D	Over-engineering	Match each workflow to the right API instead

Blocking → synchronous. Latency-tolerant → batch.

Review with fresh eyes, in passes

SELF-REVIEW FAILS

- Same session keeps its reasoning
- Less likely to question its own choices
- "Review carefully" doesn't fix it
- Extended thinking doesn't either

WHAT WORKS

- Independent instance, no generator context
- Per-file passes for local issues
- One integration pass for cross-file flow
- Confidence per finding for review routing

A pull request modifies 14 files across the stock tracking module. Your single-pass review produces inconsistent results: detailed feedback for some files, superficial comments for others, obvious bugs missed, and contradictory feedback, flagging a pattern in one file while approving identical code elsewhere. How should you restructure the review?

A. Split into focused passes: analyze each file for local issues, then run a separate integration pass on cross-file data flow.

B. Require developers to split large PRs into submissions of 3–4 files before the automated review runs.

C. Switch to a higher-tier model with a larger context window to give all 14 files adequate attention.

D. Run three independent passes on the full PR and only flag issues that appear in at least two of the three.

A · Per-file passes + an integration pass

OPTION	PATTERN	WHY IT FAILS
B	Shift the burden	Moves work to developers; the system doesn't improve
C	Bigger model	A larger window doesn't fix attention quality
D	Consensus filter	Suppresses real bugs caught intermittently

OFFICIAL PREP EXERCISE 3 · ~6 MIN

Structured data extraction pipeline

- **Schema:** required + optional fields, an `"other"` + detail enum, nullable fields; check absent fields come back `null`
- **Loop:** on validation failure, resend document + failed extraction + error; log which errors retries fix
- **Batch:** 100 docs through the Message Batches API; resubmit failures by `custom_id`; check the SLA math
- **Route:** field-level confidence → humans; accuracy by document type and field
- **Reinforces:** D4 · D5

What the exam asks

- 5.1 Manage conversation context to preserve critical information across long interactions
- 5.2 Design effective escalation and ambiguity resolution patterns
- 5.3 Implement error propagation strategies across multi-agent systems
- 5.4 Manage context effectively in large codebase exploration
- 5.5 Design human review workflows and confidence calibration
- 5.6 Preserve information provenance and handle uncertainty in multi-source synthesis

Task statements verbatim from the CCAR-F Exam Guide v1.0, Section 6

Keep the facts, drop the noise

RISK	FIX
Summaries blur amounts, dates, promises	Persistent **"case facts" block** outside the summarized history
40+ fields per order lookup, 5 matter	**Trim tool output** to the relevant fields before it piles up
Lost in the middle	Key findings **first**, details under explicit section headers
Downstream agent has a small budget	Upstream returns **structured data**, not reasoning chains
Long exploration drifts to "typical patterns"	**Scratchpad files**, subagents for discovery, `/compact`
Crash mid-run	Each agent exports state; coordinator loads a **manifest**

When to escalate

SITUATION	DO
Customer asks for a human	**Escalate now.** Don't investigate first
Frustrated, but the issue is simple	Acknowledge, offer to resolve; escalate if they ask again
Policy is silent or ambiguous	**Escalate** (e.g., competitor price match)
No meaningful progress	Escalate
Several customers match	**Ask for another identifier.** Don't guess

Unreliable triggers: sentiment scores and self-reported confidence. Neither tracks case complexity.

Your agent achieves 55% first-contact resolution, well below the 80% target. Logs show it escalates straightforward cases (standard damage replacements with photo evidence) while attempting to handle complex situations requiring policy exceptions on its own. What's the most effective way to improve escalation calibration?

A. Add explicit escalation criteria to your system prompt with few-shot examples of when to escalate versus resolve.

B. Have the agent self-report a confidence score (1–10) and route to humans when it falls below a threshold.

C. Deploy a separate classifier trained on historical tickets to predict which requests need escalation.

D. Implement sentiment analysis and escalate when negative sentiment exceeds a threshold.

A · Explicit criteria + few-shot examples

OPTION	PATTERN	WHY IT FAILS
B	Unreliable proxy	The agent is already confidently wrong on hard cases
C	Over-engineering	Labeled data + ML infra before trying the prompt
D	Unreliable proxy	Sentiment doesn't track case complexity

Hard rule → code. Judgment boundary → criteria + examples.

Errors the coordinator can recover from

PROPAGATE

- Failure type
- The query that was attempted
- Partial results
- Alternatives worth trying

ANTI-PATTERNS

- Generic "search unavailable"
- Empty result marked as success
- One failure kills the workflow
- Retrying non-retryable errors

Recover transient failures **locally**. Annotate the final report with **coverage gaps**.

The web search subagent times out while researching a complex topic. You need to design how this failure information flows back to the coordinator agent. Which error propagation approach best enables intelligent recovery?

A. Return structured error context to the coordinator: failure type, attempted query, partial results, and potential alternatives.

B. Retry with exponential backoff inside the subagent, returning a generic "search unavailable" status after all retries fail.

C. Catch the timeout within the subagent and return an empty result set marked as successful.

D. Propagate the timeout exception to a top-level handler that terminates the entire research workflow.

A · Structured error context

OPTION	PATTERN	WHY IT FAILS
B	Hide the error	Generic status removes what the coordinator needs
C	Hide the error	Failure disguised as success; no recovery possible
D	Kill the workflow	Ends everything when recovery could succeed

Humans and sources

HUMAN REVIEW

- 97% overall can hide a weak document type
- Check accuracy by type **and** field first
- Field-level confidence, calibrated on labeled data
- Stratified sampling of high-confidence output
- Low confidence or contradictions → human

PROVENANCE

- Claim → source mappings survive synthesis
- Conflicting stats: keep both, cite both
- Dates on every finding
- Separate established from contested
- Tables for financials, prose for news

OFFICIAL PREP EXERCISE 4 · ~6 MIN

Debug a multi-agent research pipeline

- **Build:** coordinator with `"Task"` in `allowedTools`; two subagents get findings in their prompts
- **Parallel:** several `Task` calls in one response; measure the latency gain
- **Provenance:** each finding = claim, excerpt, source, date; check synthesis keeps attribution
- **Break it:** simulate a timeout and conflicting statistics; check partial results and coverage gaps
- **Reinforces:** D1 · D2 · D5

SEGMENT 3 COMPLETE

Q&A

SEGMENT 4 · 1:50 → 2:00 · 10 MIN

Exam strategy and study plan

All domains

Exam guide §7 + §8

Official prep courses

Read the trigger, not the options

CUE IN THE STEM	POINTS TO
"financial", "must always", "guaranteed"	Hook or prerequisite gate
"minimal descriptions", "similar tools"	Rewrite or split tool descriptions
"every developer who clones"	`.claude/commands/`, project CLAUDE.md
"regardless of location"	`.claude/rules/` with globs
"dozens of files", "service boundaries"	Plan mode
"hangs", "waiting for input"	`-p`
"overnight", "not blocking"	Message Batches API
"inconsistent depth", "contradictory"	Per-file + integration passes
"all subagents succeeded, still incomplete"	Coordinator decomposition
"absent from the source"	Nullable field, not a retry

The official prep path

FROM THE EXAM GUIDE

- Build an Agent SDK agent: loop, tools, errors, sessions, subagents
- Configure Claude Code: hierarchy, rules, skills, one MCP server
- Design MCP tools with structured errors
- Build an extraction pipeline + batches
- Practice few-shot, criteria, multi-pass review
- Drill context and escalation patterns

FREE PREP COURSES

- Building with the Claude API
- Introduction to Model Context Protocol
- Claude Code in Action
- AI Fluency: Framework & Foundations
- Claude 101
- Claude with Amazon Bedrock · Claude on Google Cloud

Three weeks, weighted by the blueprint

WEEK	DOMAINS	DO
1	D1 · D2 (45%)	Prep exercises 1 + 4 · API + MCP courses · samples 1–3, 7–9
2	D3 · D4 (40%)	Prep exercises 2 + 3 · Claude Code course · samples 4–6, 10–12
3	D5 + review (15%)	Re-read all 30 task statements · redo all 12 samples timed

Build the four prep exercises yourself. The guide's samples are drawn from scenarios those exercises cover.

If you remember seven things

1. **Loop on `stop_reason`.**
   "tool_use" continues, "end_turn" stops. Never parse text.

2. **Subagents know only what you pass them.**
   Paste findings in. Fan out in one response.

3. **Hard rules go in hooks, not prompts.**
   Judgment calls go in criteria + examples.

4. **The description is the router.**
   Fewer, sharper tools. Structured, categorized errors.

5. **Put config at the right layer.**
   Project vs user. Rules by glob. `-p` in CI.

6. **Schema-valid isn't correct.**
   Nullable fields, targeted retries, batch what can wait.

7. **Keep facts, sources, and errors explicit.**
   Case facts, provenance, coverage gaps, human routing.

RESOURCES · Q&A

Thank you. Questions?

- Official Exam Guide v1.0 (PDF)
- Official prep courses · 7 free courses
- Building effective agents · Claude Code docs
- Course repo

Lucas Soares
O'Reilly Live Training